48 DOCKER POSTGRES + LOCAL POSTGRES — END TO END WITH NESTJS

theory/41 ran the student CRUD on Supabase (cloud). This file does the

SAME app two more ways, start to finish, with real commands:

```
PART A — Postgres in DOCKER   (a container on your PC)
PART B — Postgres INSTALLED LOCALLY on Windows (a Windows service)
PART C — ONE codebase that switches between Docker / Local / Supabase
```

The NestJS code (entity, service, controller from src/student-pg/) does NOT change between them. Only .env and the ssl setting change.

```
Your NestJS app  ->  TypeORM  ->  pg driver  ->  localhost:5432  ->  Postgres
                                                  (Docker container OR Windows service)
```

WHAT STAYS THE SAME IN ALL THREE

src/student-pg/student.entity.ts	the students table
src/student-pg/student-pg.service.ts	Repository CRUD
src/student-pg/student-pg.controller.ts	/student-pg/... routes
src/student-pg/student-pg.module.ts	forFeature([Student])
npm packages	@nestjs/typeorm typeorm pg

What changes:

Setting	Docker	Local install	Supabase
host	localhost	localhost	aws-0-...pooler.supabase.com
password	you choose in -e	you choose at setup	from Supabase dashboard
database	nest_crud	nest_crud	postgres
ssl	off	off	on
internet	not needed	not needed	needed

PART A — POSTGRES IN DOCKER, END TO END

A1 — INSTALL DOCKER DESKTOP

WHAT

docker.com → Docker Desktop for Windows → install → restart → open it. Check in a terminal:

```
docker --version
docker run hello-world
```

THEORY

```
- IMAGE      a ready-made package ("postgres:17" = Postgres 17 + Linux)
- CONTAINER  a running copy of an image, isolated from your PC
- VOLUME     storage that lives OUTSIDE the container, so data survives
             when the container is deleted
- PORT MAP   -p 5432:5432 connects your PC's port 5432 to the
             container's port 5432, so apps on your PC reach it at
             localhost:5432
Docker Desktop on Windows runs containers inside WSL2 (a small Linux
VM) — that's why it must be running before any docker command works.
```

A2 — START A POSTGRES CONTAINER

WHAT

```
  docker run --name pg-dev ^
    -e POSTGRES_USER=postgres ^
    -e POSTGRES_PASSWORD=devpass ^
    -e POSTGRES_DB=nest_crud ^
    -p 5432:5432 ^
    -v pgdata:/var/lib/postgresql/data ^
    -d postgres:17

(^ = line continuation in cmd; use ` in PowerShell, \ in Git Bash,
 or put it all on one line)
```

Each flag:

--name pg-dev	name to refer to it later (start/stop/exec)
-e POSTGRES_USER	database user created on first start
-e POSTGRES_PASSWORD	that user's password (required by the image)
-e POSTGRES_DB	database created on first start → no CREATE DATABASE needed
-p 5432:5432	PC port : container port
-v pgdata:/var/...	named volume, keeps data if the container is removed
-d	detached = runs in the background

postgres:17	image + version (pin the version, don't rely on "latest")

The POSTGRES_* variables are only used the FIRST time, when the volume is empty. Changing them later does nothing unless you delete the volume.

A3 — CHECK IT'S RUNNING

```
docker ps                                   // pg-dev should be "Up"
docker logs pg-dev                          // last line: "database system is ready to accept
connections"
docker exec -it pg-dev psql -U postgres -d nest_crud
  nest_crud=# SELECT version();
  nest_crud=# \l                            // list databases
  nest_crud=# \q                            // quit
```

docker exec runs a command INSIDE the container — psql is already installed there, so you don't need it on Windows.

A4 — SAME THING WITH docker-compose (recommended)

Put this in docker-compose.yml at the project root (next to package.json):

services:

postgres:

```
image: postgres:17
container_name: pg-dev
restart: unless-stopped
environment:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: devpass
  POSTGRES_DB: nest_crud
ports:
  - "5432:5432"
volumes:
  - pgdata:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  retries: 5
```

volumes:

```
pgdata:
```

Commands:

```
docker compose up -d        // create + start
docker compose ps           // status (shows "healthy")
docker compose stop         // stop, keep data
docker compose start        // start again
```

```
docker compose down         // remove container, KEEP volume (data)
docker compose down -v      // remove container AND volume (data gone)
```

THEORY

- the whole setup lives in one file in git → every teammate gets the exact same database with one command
- restart: unless-stopped → comes back after a PC restart
- healthcheck → pg_isready tells Docker when Postgres can accept connections (not just "the container started")

A5 — .env FOR DOCKER

```
  DATABASE_URL="postgresql://postgres:devpass@localhost:5432/nest_crud"
  DB_SSL=false

user postgres / password devpass / db nest_crud = the POSTGRES_*
values from A2/A4. Host is localhost because of the port mapping.
```

A6 — NESTJS CONFIG (app.module.ts)

```
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (configService: ConfigService) => ({
    type: 'postgres',
    url: configService.getOrThrow<string>('DATABASE_URL'),
    ssl: configService.get('DB_SSL') === 'true' ? { rejectUnauthorized: false } : false,
    autoLoadEntities: true,
    synchronize: true, // dev only
  }),
}),
```

Why ssl must be OFF here: the Docker postgres image doesn't have SSL set up. With ssl on, connecting fails with "The server does not support SSL connections". (The current project hard-codes ssl ON for Supabase — this DB_SSL switch is what lets the same code run against Docker.)

A7 — RUN THE APP AND TEST

```
docker compose up -d
npm run start:dev
```

On boot, synchronize creates the students table inside the container (theory/44). Then:

```
curl -X POST http://localhost:3000/student-pg/createstudent -H "Content-Type: application/json" -d "
{\"name\":\"Asha\",\"age\":21,\"email\":\"asha@example.com\",\"course\":\"CS\"}"
-> {"id":1,"name":"Asha","age":21,"email":"asha@example.com","course":"CS",...}

curl http://localhost:3000/student-pg/getallstudents
curl http://localhost:3000/student-pg/getstudent/1
curl -X PATCH http://localhost:3000/student-pg/updatestudent/1 -H "Content-Type: application/json" -d "
```

```
{\"age\":22}"
curl -X DELETE http://localhost:3000/student-pg/deletestudent/1
```

Look at the data directly:

```
docker exec -it pg-dev psql -U postgres -d nest_crud -c "\dt"
docker exec -it pg-dev psql -U postgres -d nest_crud -c "SELECT * FROM students;"
```

A8 — USEFUL DOCKER POSTGRES COMMANDS

Backup the database to a file:

```
  docker exec pg-dev pg_dump -U postgres nest_crud > backup.sql
Restore it:
  docker exec -i pg-dev psql -U postgres -d nest_crud < backup.sql
Start completely fresh (deletes all data):
  docker compose down -v && docker compose up -d
Open a shell inside the container:
  docker exec -it pg-dev bash
Add a GUI (pgAdmin) to docker-compose.yml:
  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@example.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports: ["5050:80"]
  -> open http://localhost:5050, add server with host "postgres"
     (the SERVICE name — containers reach each other by service name,
      not localhost), user postgres, password devpass
```

PART B — POSTGRES INSTALLED LOCALLY ON WINDOWS, END TO END

B1 — INSTALL

WHAT

postgresql.org/download/windows → EDB installer → run it:

```
- components: PostgreSQL Server, pgAdmin 4, Command Line Tools
- data directory: keep default
- password for the "postgres" superuser: choose and REMEMBER it
- port: 5432 (keep default)
- locale: default
```

THEORY

- installs Postgres as a WINDOWS SERVICE ("postgresql-x64-17") that starts automatically with Windows and listens on localhost:5432
- pgAdmin = official GUI; psql = official command-line client
- data lives in C:\Program Files\PostgreSQL\17\data

B2 — ADD psql TO PATH (so `psql` works in any terminal)

Windows search → "Edit the system environment variables" → Environment Variables → Path → New → C:\Program Files\PostgreSQL\17\bin Open a NEW terminal: psql --version

B3 — CHECK THE SERVICE IS RUNNING

PowerShell:

```
Get-Service postgresql*
Start-Service postgresql-x64-17      // if Stopped (admin terminal)
Stop-Service  postgresql-x64-17
```

or: Win+R → services.msc → postgresql-x64-17 → Start

B4 — CREATE THE DATABASE (and optionally an app user)

```
psql -U postgres -h localhost
Password for user postgres: ********

CREATE DATABASE nest_crud;

-- better practice: a separate user for the app, not the superuser
CREATE USER nest_user WITH PASSWORD 'nestpass';
GRANT ALL PRIVILEGES ON DATABASE nest_crud TO nest_user;
\c nest_crud
GRANT ALL ON SCHEMA public TO nest_user;     -- needed on Postgres 15+
\q
```

Same in pgAdmin: Servers → PostgreSQL 17 → right-click Databases → Create → Database → name: nest_crud → Save.

THEORY

- unlike Docker's POSTGRES_DB, a local install only has the default "postgres" database, so you create nest_crud yourself
- since Postgres 15 ordinary users can't create tables in the public schema by default → the GRANT ON SCHEMA public line, otherwise synchronize fails with "permission denied for schema public"

B5 — .env FOR LOCAL

As superuser:

```
  DATABASE_URL="postgresql://postgres:<your-install-password>@localhost:5432/nest_crud"
  DB_SSL=false
Or as the app user from B4:
  DATABASE_URL="postgresql://nest_user:nestpass@localhost:5432/nest_crud"
  DB_SSL=false
```

app.module.ts: the SAME config as A6 — nothing else changes.

B6 — RUN THE APP AND TEST

```
npm run start:dev
```

Same curl commands as A7. Check the data:

```
  psql -U postgres -d nest_crud -c "\dt"
  psql -U postgres -d nest_crud -c "SELECT * FROM students;"
or pgAdmin: nest_crud -> Schemas -> public -> Tables -> students ->
right-click -> View/Edit Data -> All Rows.
```

B7 — USEFUL LOCAL COMMANDS

```
Backup:   pg_dump -U postgres -d nest_crud -f backup.sql
Restore:  psql -U postgres -d nest_crud -f backup.sql
Drop and recreate (fresh start):
          psql -U postgres -c "DROP DATABASE nest_crud;"
          psql -U postgres -c "CREATE DATABASE nest_crud;"
Forgot the postgres password: edit
  C:\Program Files\PostgreSQL\17\data\pg_hba.conf, set the localhost
  lines to "trust", restart the service, run
  ALTER USER postgres PASSWORD 'newpass';  then set them back to
  "scram-sha-256" and restart again.
```

PART C — ONE CODEBASE, SWITCH BETWEEN DOCKER / LOCAL / SUPABASE

C1 — THE SWITCH IS ONLY .env

Docker:

```
  DATABASE_URL="postgresql://postgres:devpass@localhost:5432/nest_crud"
  DB_SSL=false
Local install:
  DATABASE_URL="postgresql://postgres:mypass@localhost:5432/nest_crud"
  DB_SSL=false
Supabase:
  DATABASE_URL="postgresql://postgres.<ref>:<password>@aws-0-ap-northeast-
1.pooler.supabase.com:5432/postgres"
  DB_SSL=true

Tip: keep three files (.env.docker, .env.local-pg, .env.supabase) and
copy the one you want over .env, or load by NODE_ENV:
  ConfigModule.forRoot({ isGlobal: true, envFilePath: `.env.${process.env.NODE_ENV ?? 'development'}` })
All of them stay in .gitignore; commit a .env.example with fake values.
```

C2 — DON'T RUN DOCKER AND LOCAL ON THE SAME PORT

Both want port 5432. If the Windows service is running, docker run -p 5432:5432 fails with "port is already allocated" — or worse, the app silently connects to the WRONG server.

Fix — give Docker a different PC port:

```
  ports: ["5433:5432"]        // PC 5433 -> container 5432
  DATABASE_URL="postgresql://postgres:devpass@localhost:5433/nest_crud"
Or stop one of them:
  Stop-Service postgresql-x64-17     /    docker compose stop
```

Check what's using 5432:

```
Get-NetTCPConnection -LocalPort 5432 | Select OwningProcess
Get-Process -Id <pid>
```

C3 — WHICH ONE WHEN

Option	Best for	Downsides
Docker	daily dev, teams, fresh resets	needs Docker Desktop running
Local install	offline, pgAdmin users, simple	permanent service, version upgrades by hand
Supabase	sharing, deployed apps, demos	needs internet, slower, free-tier limits

Common setup: Docker (or local) for development, a hosted database (Supabase / RDS) for production — same code, different .env.

TROUBLESHOOTING

```
ECONNREFUSED 127.0.0.1:5432
    server not running -> docker compose ps / Get-Service postgresql*
password authentication failed for user "postgres"
    wrong password in DATABASE_URL; for Docker, POSTGRES_PASSWORD only
    applies on FIRST start -> docker compose down -v and start again
database "nest_crud" does not exist
    local: CREATE DATABASE nest_crud;  Docker: set POSTGRES_DB, recreate volume
The server does not support SSL connections
    DB_SSL=false for Docker/local
permission denied for schema public
    GRANT ALL ON SCHEMA public TO nest_user;  (Postgres 15+)
port is already allocated / bind: address already in use
    local service and Docker both on 5432 -> see C2
docker: command not found / error during connect
    Docker Desktop isn't running
'psql' is not recognized
    add C:\Program Files\PostgreSQL\17\bin to PATH (B2), new terminal
table data disappeared
    Docker container removed without a volume, or `down -v` was used
```

★ RULE OF THUMB

Docker: one docker-compose.yml, `docker compose up -d`, database ready with POSTGRES_DB. Local: install, create nest_crud yourself, keep the service running. Both live on localhost:5432 with SSL off. The NestJS

entity, service and controller never change — only DATABASE_URL and DB_SSL decide whether you're talking to Docker, the local install or Supabase.

See also: **theory/42-ways-to-connect-postgres.txt** (all hosts and libraries), **theory/41-supabase-postgres-crud-full-steps.txt** (the Supabase build), **theory/40-postgres-end-to-end.txt**, **theory/44-how-the-table-was-created.txt** (synchronize), **theory/28-environment-variables.txt**